Section 8 : Statements and Operators

Informations sur le papier

- Titre : Programmation C++
- Auteurs : `Etienne KOA`

Aperçu de la section

- Expressions, instructions et opérateurs

  +Expressions

  - Déclarations et instructions de bloc
  - Opérateurs
    * Mission
    * Arithmétique
    * Incrémenter et décrémenter
    * Égalité
    * Relationnel
    * Logique
    * Mission composée
    * Précédent

Expressions et déclarations

+Expressions

- Une expression est :
  - L'élément de base d'un programme
  - `une séquence d'opérateurs et d'opérandes qui spécifie un calcul`
  - Calcule une valeur à partir d'un certain nombre d'opérandes
  - Il y a bien plus dans les expressions - ce n'est pas nécessaire à ce niveau.
- Expressions - exemples
- déclarations
  - Une déclaration est :
    * Une ligne complète de code qui effectue une action.
    * Généralement terminé par un point-virgule
    * Contient généralement des expressions
    * `C++` a de nombreux types d'instructions
      · `expression, null, composé, sélection, itération, déclaration, saut, blocs d'essai et blocs`.

```
34                    // literal

favorite_number           // variable

1.5 + 2.8             // addition

2 * 5                     // multiplication

a > b                     // relational

a = b                     // assignment
```

Figure 1: Expressions

- déclarations - exemples

```
int x;                    // declaration

favorite_number = 12;     // assignment

1.5 + 2.8;                    // expression

x = 2 * 5;                    // assignment

if (a > b) cout << "a is greater than b";    // if

;
```

Figure 2: déclarations

Utilisation des opérateurs

- `C++` possède un riche ensemble d'opérateurs
  - unaire, binaire, ternaire
- Les opérateurs courants peuvent être regroupés comme suit :
  - Mission
  - Arithmétique
  - Incrémenter/décrémenter

  - Égalité
  - Relationnel
  - Logique
  - accès membre
  - Autre

Opérateur d'affectation (=)

```
lhs = rhs
```

- `rhs` est une expression qui est évaluée à une valeur.
- La valeur du `rhs` est stockée dans le `lhs`.
- La valeur du `rhs` doit être de type compatible avec le `lhs`.
- Le `lhs` doit être assignable.
- L'expression d'affectation est évaluée par rapport à ce qui a été attribué par ujsut.
- Plus d'une variable peut être affectée dans une seule instruction.

Expressions et conversions mixtes

1) Conversions

- Les types « Supérieur » et « Lower » sont basés sur la taille des valeurs que les types peuvent contenir.
  - `Long double, double, float, unsigned long, long, unsigned int, int`.
  - Les types `short` et `char` sont toujours convertis en `int`.
- `Type Coercion :` conversion d'un opérande vers un autre type de données.
- `Promotion :` conversion vers un type supérieur
  - Utilisé dans les expressions mathématiques.
- `Dégradation :` conversion vers un type inférieur.
  - Utilisé avec une affectation au type inférieur.

2) Expressions de types mixtes

- Exemples
- Casting de type explicite - `static_cast<type>`

Test d'égalité

- Les opérateurs `==` et `!=`.
  - Compare les valeurs de 2 expressions.
  - Évalue à un « booléen (vrai ou faux, 1 ou 0) »
  - Couramment utilisé dans les instructions de flux de contrôle.

- `lower` op `higher` the `lower` is promoted to a `higher`

  `2 * 5.2`

  2 is promoted to 2.0

- `lower = higher;` the `higher` is demoted to a `lower`

```
int num {0};
num  = 100.2;
```

Figure 3: Expressions de types mixtes

```
int total_amount {100};
int total_number {8};
double average {0.0};

average = total_amount / total_number;
cout << average << endl;            // displays 12

average = static_cast<double>(total_amount) / total_number;
cout << average << endl;            // displays 12.5
```

Figure 4: Casting de type explicite

```
expr1 == expr2

expr1 != expr2

100 == 200
num1 != num2
```

Figure 5: Test d'égalité

```
bool result {false};

result = (100 == 50+50);

result = (num1 != num2);

cout << (num1 == num2) << endl;    // 0 or 1
cout << std::boolalpha;
cout << (num1 == num2) << endl;    // true or false
cout << std::noboolalpha;
```

Figure 6: Test d'égalité

Opérateurs relationnels

`expr1` **op** `expr2`

Operator	Meaning
>	greater than
>=	greater than or equal to
<	less than
<=	less than or equal to
<=>	three-way comparison (C++20)

Figure 7: Opérateurs relationnels

Opérateurs logiques

- `not (!)`
- `and (&&)`
- `or (||)`
- Priorité
  - `not` a une priorité plus élevée que `and`
  - `and` a une priorité plus élevée que `or`
  - `not` est un opérateur unaire

Operator	Meaning
`not` `!`	negation
`and` `&&`	logical and
`or` `||`	logical or

Figure 8: Opérateurs logiques

expression `a`	`not a` `!a`
true	false
false	true

Figure 9: Opérateurs logiques

expression `a`	expression `b`	`a and b` `a && b`
true	true	**true**
true	false	false
false	true	false
false	false	false

Figure 10: Opérateurs logiques

expression a	expression b	a or b a || b
true	true	**true**
true	false	**true**
false	true	**true**
false	false	false

Figure 11: Opérateurs logiques

  - Opérateurs binaires `and` et `or`

- Exemples

```
num1 >= 10 && num1 < 20
num1 <= 10 || num1 >=20

!is_raining && temperature > 32.0

is_raining || is_snowing

temperature > 100 && is_humid || is_raining
```

Figure 12: Opérateurs logiques

- Évaluation des courts-circuits
  - Lors de l'évaluation d'une expression logique `C++` s'arrête dès que le résultat est connu.

Opérateurs d'affectation de composés

- `op=`
- `Exemples`

Priorité des opérateurs (liste non complète)

- De haut en bas
- Qu'est-ce que l'associativité ?

```
expr1 && expr2 && expr3

expr1 || expr2 || expr3
```

Figure 13: Opérateurs logiques

Operator	Example	Meaning
+=	lhs += rhs;	lhs = lhs + (rhs);
-=	lhs -= rhs;	lhs = lhs - (rhs);
*=	lhs *= rhs;	lhs = lhs * (rhs);
/=	lhs /= rhs;	lhs = lhs / (rhs);
%=	lhs %= rhs;	lhs = lhs % (rhs);
>>=	lhs >>= rhs;	lhs = lhs >> (rhs);
<<=	lhs <<= rhs;	lhs = lhs << (rhs);
&=	lhs &= rhs;	lhs = lhs & (rhs);
^=	lhs ^= rhs;	lhs = lhs ^ (rhs);
|=	lhs |= rhs;	lhs = lhs | (rhs);

Figure 14: Opérateurs d'affectation de comtés

```
lhs op= rhs; // lhs = lhs op (rhs);

a += 1;        // a = a + 1;
a /= 5;        // a = a / 5;
a *= b + c;    // a = a * (b + c);

cost += items * tax;
               //cost = cost + (items * tax);
```

Figure 15: Opérateurs d'affectation de comtés

Operator	Associativity
[] -> . ()	left to right
++ -- not -(unary) *(de-ref) & sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
|	left to right
&&	left to right
||	left to right
= op= ?:	right to left

Figure 16: Précédence de l'opérateur

  - Utiliser des règles de priorité lorsque les opérateurs adjacents sont différents

  `expr1 op1 expr2 op2 expr3 // priorité`

- Utiliser des règles d'associativité lorsque les opérateurs adjacents ont la même priorité

  `expr1 op1 expr2 op1 expr3 // associativité`

- Utilisez des parenthèses pour lever absolument tout doute.
- Exemple :

```
result = num1 + num2 * num3;
result = ( num1 + (num2 * num3) );

result1 = num1 + num2 - num3;
result1 = ( (num1 + num2) - num3 );
```

Figure 17: Précédence de l'opérateur